BIG DATA TECHNOLOGY LAB (BDTL) · PRACTICAL REPORT

Bucket 10 · Apache Spark Experiments 1 and 3

Submitted by

Adesh Patankar

Experiment	Title and learning objective
Experiment 1	**Load and Preprocess Data** Write a PySpark script to read a text file into an RDD or DataFrame and clean data (trim, lowercase).
Experiment 3	**Store Results and Visualize Output** Save word count results to a text file or DataFrame and visualize using show() or export to CSV.

Platform: Apache Spark 3.5.8 (Scala 2.12, Java 8, local mode) on Apache Hadoop 3.3.6 (pseudo-distributed HDFS + YARN), Ubuntu 24.04 LTS.
All execution evidence in this report consists of screenshots of the actual terminal session in which the experiments were run.

Contents

	Section
Experiment 1	**Load and Preprocess Data**
	A. Experiment Details B. Aim C. Theory D. Requirements E. Procedure and Implementation F. Execution Screenshots G. Output and Results H. Result I. Observations and Troubleshooting
Experiment 3	**Store Results and Visualize Output**
	A. Experiment Details B. Aim C. Theory D. Requirements E. Procedure and Implementation F. Execution Screenshots G. Output and Results H. Result I. Observations and Troubleshooting

Bucket 10 – Experiment 1: Load and Preprocess Data

A. Experiment Details

Field	Details
Bucket	10
Sub-experiment No.	1
Title	Load and Preprocess Data
Description / Learning Objective (bucket list)	Write a PySpark script to read a text file into an RDD or DataFrame and clean data (trim, lowercase).
Student	Adesh Patankar
Subject	Big Data Technology Lab (BDTL)
Tool / Technology	PySpark (Apache Spark 3.5.8, Python 3.12) with Apache Hadoop 3.3.6 HDFS

B. Aim

To read a text file stored in HDFS into a PySpark **RDD** and a **DataFrame**, and to clean it by trimming spaces, converting to lowercase, removing punctuation and dropping empty lines.

C. Theory

C.1 PySpark

PySpark is the Python API of Apache Spark. A Python driver program creates a `SparkSession` (and its `SparkContext`); Spark's JVM engine executes the job, and Python functions passed to transformations (lambdas, helper functions) run in Python worker processes next to the executors. Data can be handled as an **RDD** (a distributed collection of Python objects, processed with functions such as `map`, `flatMap`, `filter`, `reduceByKey`) or as a **DataFrame** (a distributed table with named columns, processed with built-in column functions that Spark optimises). Transformations are lazy; actions such as `count`, `collect`, `show` and `save…` run the job. Programs are submitted with `spark-submit`; `--py-files` ships extra Python modules to the workers.

C.2 Why preprocess before counting

Raw text is inconsistent: `Spark`, `SPARK` and `Spark.` are the same word but would be counted as three different keys, and blank lines or extra spaces produce empty tokens. Cleaning normalises the text so that later analysis (Experiment 2) is correct.

Cleaning step	RDD (Python function in `clean_line`)	DataFrame (built-in column functions)
Load	`sc.textFile(path)` – one string per line	`spark.read.text(path)` – one column `value`
Trim + lowercase	`line.strip().lower()`	`F.trim(F.lower(col))`
Remove punctuation	`re.sub(r"[^a-z0-9\s]", " ", line)`	`F.regexp_replace(col, "[^a-z0-9 ]", " ")`
Collapse spaces	`" ".join(line.split())`	`F.regexp_replace(col, " +", " ") + trim`
Drop empty lines	`rdd.filter(lambda l: l != "")`	`df.filter(F.col("clean") != "")`

The RDD version runs Python code for every line; the DataFrame version uses Spark's built-in functions, which run inside the JVM and are optimised by the Catalyst optimiser.

D. Requirements

Item	Used in this experiment
Host machine	Apple MacBook Air (Apple Silicon, arm64) running macOS
Virtual machine	Multipass VM `bigdata` – Ubuntu 24.04 LTS (arm64), 2 vCPU, 4 GB RAM, 2 GB swap
Java	OpenJDK 8 (1.8.0_504), `JAVA_HOME=/usr/lib/jvm/java-8-openjdk-arm64`
Spark	Apache Spark 3.5.8, pre-built for Hadoop 3 (`spark-3.5.8-bin-hadoop3`), Scala 2.12.18, installed in `/home/ubuntu/spark-3.5.8-bin-hadoop3`; run in local mode (`master = local[*]`)
Hadoop	Apache Hadoop 3.3.6, pseudo-distributed: HDFS at `hdfs://localhost:9000` and YARN
Python	Python 3.12.3 (Ubuntu 24.04), `PYSPARK_PYTHON=python3`; only the standard library (`re`, `unittest`) is used besides PySpark
Input	`bucket10/data/input.txt` uploaded to `hdfs:///bdt/bucket10/input.txt`
Runner	`bdt/bucket10/run_bucket10.sh` – 7 parts; each program is printed with `cat -n` and run with `spark-submit --py-files py/wordcount_lib.py <program>.py`

E. Procedure and Implementation

Step 1 – Input file, upload to HDFS and the shared function module

The input file was written for this bucket with deliberately *messy* text: mixed case (`FAST`, `SPARK`, `BIG`), punctuation (`, ! : ; . ...`), leading and trailing spaces, an empty line and a line containing only spaces – 7 lines in total (242 bytes). `cat -A` marks each line end with `$`, which makes the trailing spaces visible.

bucket10/data/input.txt

```
   Apache Spark is FAST, and Spark is EASY to use!
Big Data needs big tools: Spark, Hadoop and Hive.
```

```
  PySpark lets Python developers use Spark.
SPARK processes data in memory; Hadoop stores data in HDFS.

Big data... BIG insights!
```

Shell

```
cat -A data/input.txt
hdfs dfs -mkdir -p /bdt/bucket10
hdfs dfs -put -f data/input.txt /bdt/bucket10/
hdfs dfs -ls /bdt/bucket10
```

The cleaning and counting logic is kept in one module, `wordcount_lib.py`, so that the same functions are used by the programs and by the unit tests.

bucket10/py/wordcount_lib.py

```
# Bucket 10 | wordcount_lib.py - reusable (and unit-tested) word-count functions
import re

def clean_line(line):
    """Trim, lowercase and replace punctuation with spaces."""
    line = line.strip().lower().replace("'", "")
    line = re.sub(r"[^a-z0-9\s]", " ", line)
    return " ".join(line.split())           # collapse repeated spaces

def split_words(line):
    """Clean a line and split it into words."""
    return clean_line(line).split()

def preprocess(lines_rdd):
    """Clean every line and drop lines that are empty after cleaning."""
    return lines_rdd.map(clean_line).filter(lambda l: l != "")

def count_words(lines_rdd):
    """Word count with RDD transformations: flatMap -> map -> reduceByKey."""
    return (lines_rdd.flatMap(split_words)
                     .map(lambda w: (w, 1))
                     .reduceByKey(lambda a, b: a + b))
```

Step 2 – Preprocessing program

bucket10/py/preprocess.py

```
# Bucket 10 | Sub-Exp 1 : load a text file into an RDD and a DataFrame and clean it (trim,
lowercase)
from pyspark.sql import SparkSession
from pyspark.sql import functions as F
from wordcount_lib import preprocess

spark = SparkSession.builder.appName("Bucket10-Preprocess").getOrCreate()
sc = spark.sparkContext
PATH = "hdfs://localhost:9000/bdt/bucket10/input.txt"

# --- RDD ---
raw = sc.textFile(PATH)
print("RDD - raw lines   :", raw.count())
for l in raw.collect():
    print("   ", repr(l))
clean = preprocess(raw)
print("RDD - clean lines :", clean.count())
for l in clean.collect():
    print("   ", repr(l))

# --- DataFrame ---
```

```
df = spark.read.text(PATH)                                    # one column: value
clean_df = (df.withColumn("clean", F.trim(F.lower(F.col("value"))))
              .withColumn("clean", F.regexp_replace("clean", "[^a-z0-9 ]", " "))
              .withColumn("clean", F.trim(F.regexp_replace("clean", " +", " ")))
              .filter(F.col("clean") != ""))
print("DataFrame - raw rows:", df.count(), "  clean rows:", clean_df.count())
clean_df.select("clean").show(truncate=False)
spark.stop()
```

Command

```
spark-submit --py-files py/wordcount_lib.py py/preprocess.py
```

F. Execution Screenshots

Screenshots of the Ubuntu VM terminal (`ubuntu@bigdata`) while `run_bucket10.sh` was executing. Only empty margins were trimmed.

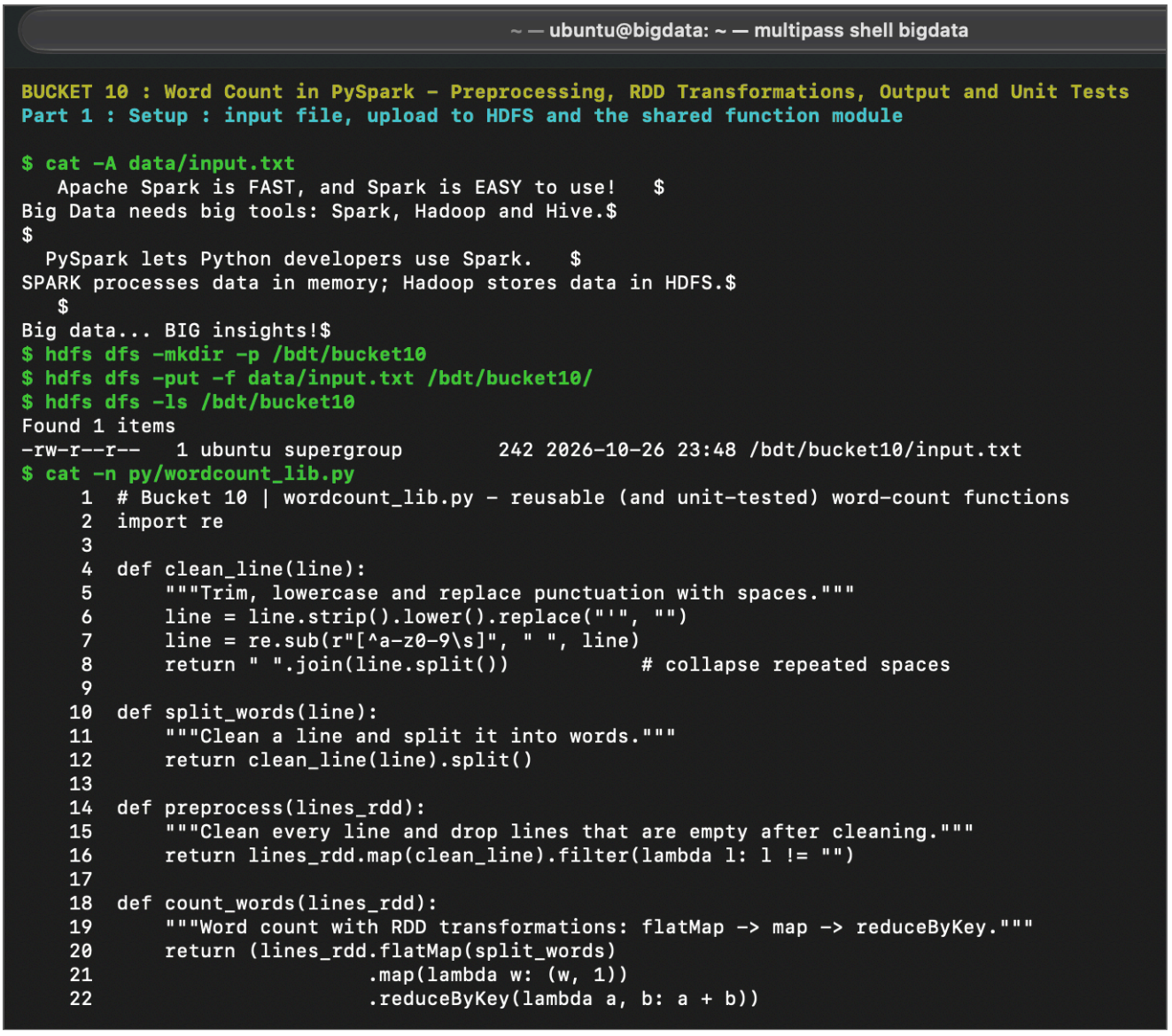


Figure 1.1: Input file, upload to HDFS and the shared module `wordcount_lib.py`

`cat -A` shows the 7 raw lines with their trailing spaces, the empty line and the whitespace-only line. The file (242 B) is uploaded to `/bdt/bucket10`, and the module with `clean_line`, `split_words`, `preprocess` and `count_words` is printed.

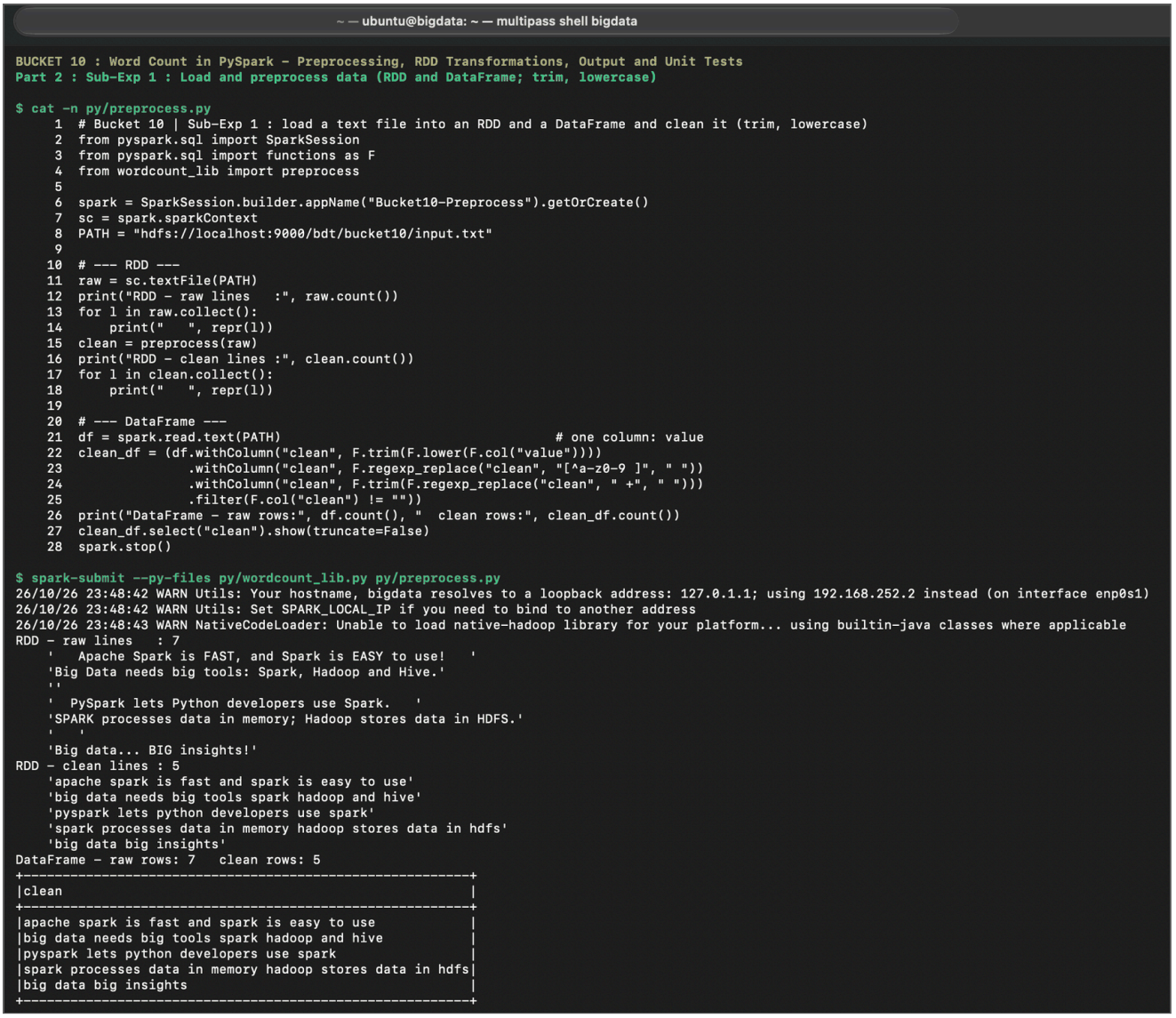


Figure 1.2: Loading and cleaning the text with an RDD and a DataFrame

The RDD contains **7 raw lines**, printed with `repr()` so that leading/trailing spaces and the empty strings are visible. After `preprocess` **5 clean lines** remain, all lowercase without punctuation or extra spaces. The DataFrame version gives the same result (*raw rows: 7, clean rows: 5*), displayed with `show(truncate=False)`.

G. Output and Results

Raw line	Clean line
`' Apache Spark is FAST, and Spark is EASY to use! '`	`apache spark is fast and spark is easy to use`
`'Big Data needs big tools: Spark, Hadoop and Hive.'`	`big data needs big tools spark hadoop and hive`
`''`	*removed*
`' PySpark lets Python developers use Spark. '`	`pyspark lets python developers use spark`
`'SPARK processes data in memory; Hadoop stores data in HDFS.'`	`spark processes data in memory hadoop stores data in hdfs`
`' '`	*removed*
`'Big data... BIG insights!'`	`big data big insights`

RDD and DataFrame produced identical output: 7 lines in, 5 clean lines out.

H. Result

> The text file was read from HDFS into both an RDD and a DataFrame and cleaned: spaces trimmed, text lowercased, punctuation removed and the 2 blank lines dropped, leaving 5 normalised lines ready for word counting.

I. Observations and Troubleshooting

Observations

- The warnings about the hostname resolving to a loopback address and the native-hadoop library are harmless in this single-node setup.
- `repr()` is useful for checking cleaning, because ordinary printing hides trailing spaces.
- Punctuation is replaced by a space rather than deleted, so text such as `data...BIG` would still split into two words.
- `wordcount_lib.py` is passed with `--py-files` so that the Python workers can import it, not just the driver.

Troubleshooting

- The programs ran without errors; no troubleshooting was required.

Bucket 10 – Experiment 3: Store Results and Visualize Output

A. Experiment Details

Field	Details
Bucket	10
Sub-experiment No.	3
Title	Store Results and Visualize Output
Description / Learning Objective (bucket list)	Save word count results to a text file or DataFrame and visualize using show() or export to CSV.
Student	Adesh Patankar
Subject	Big Data Technology Lab (BDTL)
Tool / Technology	PySpark (Apache Spark 3.5.8, Python 3.12) with Apache Hadoop 3.3.6 HDFS

B. Aim

To store the word-count results in HDFS as a text file and as a CSV file, to convert them into a DataFrame and display them with `show()`, and to visualise the most frequent words as a simple bar chart.

C. Theory

C.1 PySpark

PySpark is the Python API of Apache Spark. A Python driver program creates a `SparkSession` (and its `SparkContext`); Spark's JVM engine executes the job, and Python functions passed to transformations (lambdas, helper functions) run in Python worker processes next to the executors. Data can be handled as an **RDD** (a distributed collection of Python objects, processed with functions such as `map`, `flatMap`, `filter`, `reduceByKey`) or as a **DataFrame** (a distributed table with named columns, processed with built-in column functions that Spark optimises). Transformations are lazy; actions such as `count`, `collect`, `show` and `save…` run the job. Programs are submitted with `spark-submit`; `--py-files` ships extra Python modules to the workers.

C.2 Ways of storing and viewing results

Method	Code	Result
Text file	`rdd.saveAsTextFile(dir)`	Directory with one `part-NNNNN` per partition; each element written as `str()`, e.g. `('spark', 5)`
DataFrame	`spark.createDataFrame(rdd, ["word", "count"])`	Table with named columns; types inferred (string, long)
show()	`df.show(10)`	Prints the first 10 rows as a formatted table
CSV export	`df.coalesce(1).write.option("header", True).csv(dir)`	`coalesce(1)` reduces to one partition, so one CSV file with a header is written
Local copy	`hdfs dfs -getmerge dir file.csv`	Merges the part files of an HDFS directory into one local file (e.g. to open in Excel)

A result is a *directory* in HDFS because every task writes its own part file in parallel; an empty `_SUCCESS` file marks that the job completed. The bar chart is drawn in the terminal by repeating `#` once per occurrence.

D. Requirements

Item	Used in this experiment
Host machine	Apple MacBook Air (Apple Silicon, arm64) running macOS
Virtual machine	Multipass VM `bigdata` – Ubuntu 24.04 LTS (arm64), 2 vCPU, 4 GB RAM, 2 GB swap
Java	OpenJDK 8 (1.8.0_504), `JAVA_HOME=/usr/lib/jvm/java-8-openjdk-arm64`
Spark	Apache Spark 3.5.8, pre-built for Hadoop 3 (`spark-3.5.8-bin-hadoop3`), Scala 2.12.18, installed in `/home/ubuntu/spark-3.5.8-bin-hadoop3`; run in local mode (`master = local[*]`)
Hadoop	Apache Hadoop 3.3.6, pseudo-distributed: HDFS at `hdfs://localhost:9000` and YARN
Python	Python 3.12.3 (Ubuntu 24.04), `PYSPARK_PYTHON=python3`; only the standard library (`re`, `unittest`) is used besides PySpark
Input	`bucket10/data/input.txt` uploaded to `hdfs:///bdt/bucket10/input.txt`
Runner	`bdt/bucket10/run_bucket10.sh` – 7 parts; each program is printed with `cat -n` and run with `spark-submit --py-files py/wordcount_lib.py <program>.py`

E. Procedure and Implementation

Step 1 – Input file, upload to HDFS and the shared function module

The input file was written for this bucket with deliberately *messy* text: mixed case (`FAST`, `SPARK`, `BIG`), punctuation (`, ! : ; . ...`), leading and trailing spaces, an empty line and a line

containing only spaces – 7 lines in total (242 bytes). `cat -A` marks each line end with `$`, which makes the trailing spaces visible.

bucket10/data/input.txt

```
   Apache Spark is FAST, and Spark is EASY to use!
Big Data needs big tools: Spark, Hadoop and Hive.

  PySpark lets Python developers use Spark.
SPARK processes data in memory; Hadoop stores data in HDFS.

Big data... BIG insights!
```

Shell

```
cat -A data/input.txt
hdfs dfs -mkdir -p /bdt/bucket10
hdfs dfs -put -f data/input.txt /bdt/bucket10/
hdfs dfs -ls /bdt/bucket10
```

The cleaning and counting logic is kept in one module, `wordcount_lib.py`, so that the same functions are used by the programs and by the unit tests.

bucket10/py/wordcount_lib.py

```
# Bucket 10 | wordcount_lib.py - reusable (and unit-tested) word-count functions
import re

def clean_line(line):
    """Trim, lowercase and replace punctuation with spaces."""
    line = line.strip().lower().replace("'", "")
    line = re.sub(r"[^a-z0-9\s]", " ", line)
    return " ".join(line.split())           # collapse repeated spaces

def split_words(line):
    """Clean a line and split it into words."""
    return clean_line(line).split()

def preprocess(lines_rdd):
    """Clean every line and drop lines that are empty after cleaning."""
    return lines_rdd.map(clean_line).filter(lambda l: l != "")

def count_words(lines_rdd):
    """Word count with RDD transformations: flatMap -> map -> reduceByKey."""
    return (lines_rdd.flatMap(split_words)
                     .map(lambda w: (w, 1))
                     .reduceByKey(lambda a, b: a + b))
```

Step 2 – Store and visualise program

bucket10/py/store_visualize.py

```
# Bucket 10 | Sub-Exp 3 : store the word-count results (text file, DataFrame, CSV) and visualise
them
from pyspark.sql import SparkSession
from pyspark.sql import functions as F
from wordcount_lib import count_words

spark = SparkSession.builder.appName("Bucket10-Store-Visualize").getOrCreate()
sc = spark.sparkContext
OUT = "hdfs://localhost:9000/bdt/bucket10"

counts = count_words(sc.textFile(OUT + "/input.txt")).sortBy(lambda p: (-p[1], p[0]))

# 1. save as a text file
```

```
counts.saveAsTextFile(OUT + "/wordcount_text")
print("Saved text file :", OUT + "/wordcount_text")

# 2. convert to a DataFrame and display with show()
df = spark.createDataFrame(counts, ["word", "count"]).orderBy(F.desc("count"), "word")
df.show(10)

# 3. export the DataFrame to CSV (one file, with header)
df.coalesce(1).write.mode("overwrite").option("header", True).csv(OUT + "/wordcount_csv")
print("Saved CSV       :", OUT + "/wordcount_csv")

# 4. simple text bar chart of the top 8 words
print("\nTop words (bar chart):")
for row in df.limit(8).collect():
    print(f"  {row['word']:<10}| {'#' * row['count']} {row['count']}")
spark.stop()
```

Command

```
spark-submit --py-files py/wordcount_lib.py py/store_visualize.py
```

Step 3 – Check the stored output

Shell

```
hdfs dfs -ls /bdt/bucket10/wordcount_text /bdt/bucket10/wordcount_csv
hdfs dfs -cat /bdt/bucket10/wordcount_text/part-* | head -12
hdfs dfs -getmerge /bdt/bucket10/wordcount_csv output/wordcount.csv
ls -l output/wordcount.csv; head -12 output/wordcount.csv
```

F. Execution Screenshots

Screenshots of the Ubuntu VM terminal (`ubuntu@bigdata`) while `run_bucket10.sh` was executing. Only empty margins were trimmed.

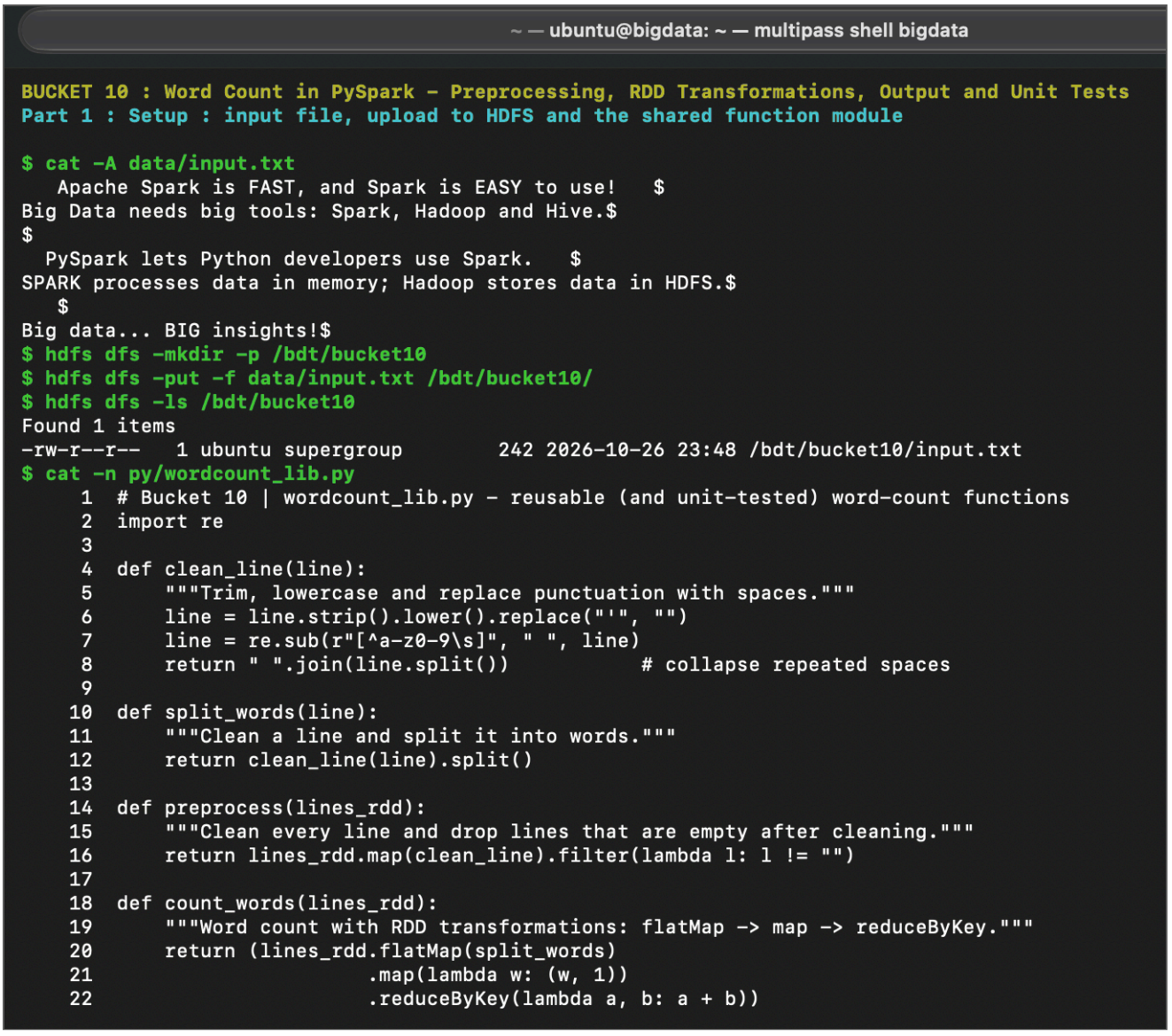


Figure 3.1: Input file, upload to HDFS and the shared module `wordcount_lib.py`

`cat -A` shows the 7 raw lines with their trailing spaces, the empty line and the whitespace-only line. The file (242 B) is uploaded to `/bdt/bucket10`, and the module with `clean_line`, `split_words`, `preprocess` and `count_words` is printed.

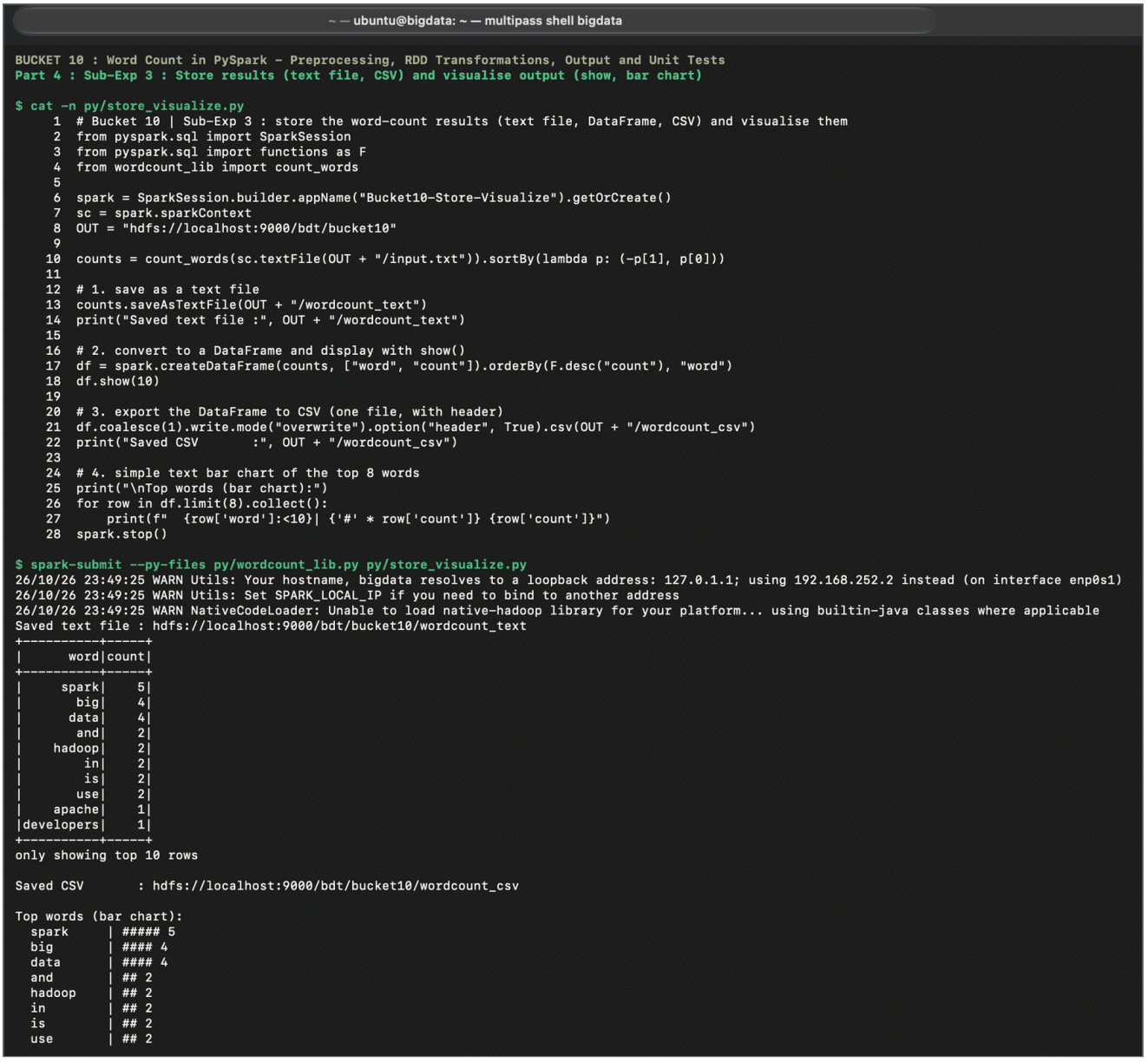


Figure 3.2: Saving the results, show() and the text bar chart

The sorted counts are saved to `wordcount_text`. The DataFrame is displayed with `show(10)` (spark 5, big 4, data 4, and/hadoop/in/is/use 2, apache 1, developers 1 – *only showing top 10 rows*), exported to `wordcount_csv`, and the top 8 words are drawn as a bar chart (`spark | ##### 5` …).

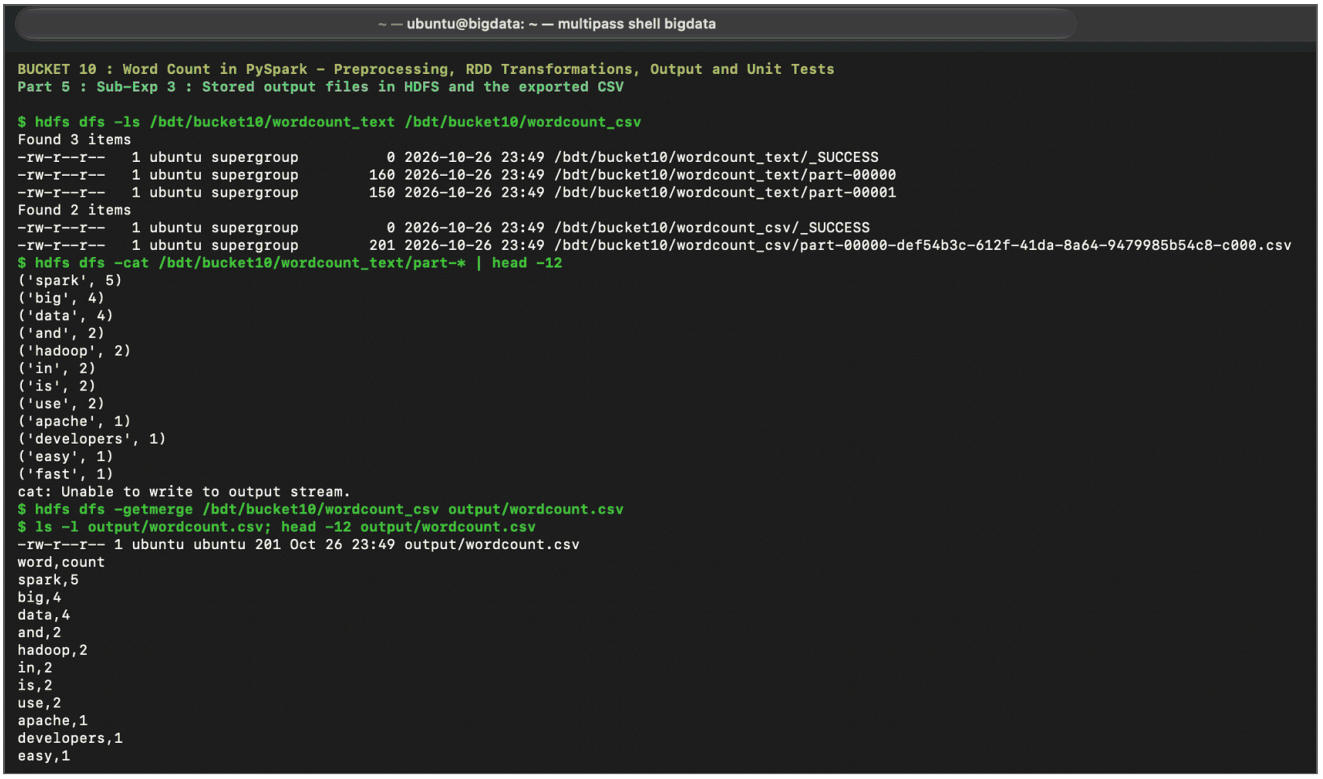


Figure 3.3: Stored output files in HDFS and the exported CSV

`wordcount_text` contains `_SUCCESS` and two part files (160 B and 150 B) with Python tuples such as `('spark', 5)`; `wordcount_csv` contains a single CSV file (201 B) because of `coalesce(1)`. The CSV is copied to `output/wordcount.csv` with `getmerge` and starts with the header `word,count`.

G. Output and Results

Output	Location	Content
Text file	`hdfs:///bdt/bucket10/wordcount_text`	_SUCCESS + part-00000 (160 B) + part-00001 (150 B); 24 lines `('word', count)`, sorted by count
DataFrame	in memory	24 rows × 2 columns (word, count); top 10 shown with show()
CSV	`hdfs:///bdt/bucket10/wordcount_csv`	_SUCCESS + one part file (201 B) with header `word,count`
Local CSV	`bucket10/output/wordcount.csv`	201 B: word,count / spark,5 / big,4 / data,4 / ...
Bar chart	terminal	spark #####, big ####, data ####, and/hadoop/in/is/use ##

H. Result

> The word-count results were stored in HDFS both as a text file and as a single CSV file with a header, displayed as a DataFrame with show(), exported to a local CSV file, and visualised as a bar chart of the most frequent words.

I. Observations and Troubleshooting

Observations

- The warnings about the hostname resolving to a loopback address and the native-hadoop library are harmless in this single-node setup.
- `saveAsTextFile` writes Python's string form of each tuple; the CSV writer produces a cleaner, tool-friendly format.
- `coalesce(1)` is fine for small results but sends all data to one task; it should be avoided for very large outputs.
- The line *cat: Unable to write to output stream* in Figure 3.3 is harmless: `head -12` closed the pipe after 12 lines while `hdfs dfs -cat` was still writing.

Troubleshooting

- The programs ran without errors; no troubleshooting was required.